Tecnológico Nacional de México
Campus Chilpancingo

Ingeniería en sistemas Computacionales

Materia:

Inteligencia Artificial

PROYECTO

Prediccion del Nivel de HbA1c
Random Forest

Integrantes :

Wendy Lizeth Morales Calvario
Sandra Bianey Moctezuma Mónico

Grupo
S8A

Docente:
Yanet Evangelista Alcocer

1. Introducción

Este proyecto desarrolla un modelo de Machine Learning supervisado para predecir el nivel de hemoglobina glucosilada (HbA1c) en pacientes, a partir de variables clínicas y demográficas básicas. El dataset utilizado es diabetes_clean.csv, que contiene 100,000 registros reales de pacientes.

La HbA1c es el indicador de referencia para diagnosticar y monitorear la diabetes mellitus. A diferencia de la glucosa en ayuno, que refleja el nivel de azúcar en un momento puntual, el HbA1c refleja el promedio del control glucémico de los últimos 2 a 3 meses, lo que lo convierte en una medida mucho más estable y confiable.

¿Por qué predecir el HbA1c?

Un análisis de HbA1c en laboratorio puede tomar días y tiene un costo que lo hace inaccesible en contextos con recursos limitados. Un modelo que estime este valor a partir de datos básicos disponibles en consulta podría ayudar a priorizar la atención de pacientes en riesgo, especialmente en clínicas rurales o con alta demanda.

Los rangos de diagnóstico reconocidos por la ADA (American Diabetes Association) son:

Valor de HbA1c	Diagnóstico
< 5.7%	Normal
5.7% – 6.4%	Prediabetes
≥ 6.5%	Diabetes

2. Objetivo

El objetivo principal es construir un modelo Random Forest Regressor que sea capaz de predecir el nivel de HbA1c de un paciente antes de realizar un análisis de laboratorio. Con esto se busca:

- Identificar qué variables clínicas influyen más en el HbA1c.
- Predecir si un paciente podría encontrarse en zona de prediabetes o diabetes.
- Demostrar que Random Forest puede capturar relaciones complejas en datos médicos reales con buena precisión.

3. Librerías utilizadas

Se utilizaron las siguientes librerías estándar del ecosistema de ciencia de datos en Python:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_squared_error

print('✓ Librerías importadas correctamente')
```

Cada librería cumple un rol específico: pandas maneja la carga y manipulación del dataset, scikit-learn proporciona el modelo y las herramientas de evaluación, y matplotlib/seaborn generan las visualizaciones de resultados.

4. Descripción del Dataset

El archivo diabetes_clean.csv contiene 100,000 registros de pacientes con 9 variables clínicas y demográficas. Cada fila representa un paciente distinto. La variable objetivo a predecir es HbA1c_level.

Variable	Tipo	Descripción
age	Numérica (años)	Edad del paciente
bmi	Numérica (kg/m²)	Índice de masa corporal
hypertension	Binaria (0/1)	Tiene hipertensión: 0=No, 1=Sí
heart_disease	Binaria (0/1)	Tiene enfermedad cardíaca: 0=No, 1=Sí
diabetes	Binaria (0/1)	Diagnóstico previo de diabetes: 0=No, 1=Sí
smoking_history	Categórica (codif.)	Historial de tabaquismo, convertida a número
gender	Categórica (codif.)	Género del paciente, convertida a número
blood_glucose_level	Numérica (mg/dL)	**Nivel de glucosa en sangre — variable más predictiva**

Codificación de variables categóricas

Random Forest, como todos los modelos de scikit-learn, solo acepta entradas numéricas. Las columnas gender y smoking_history contienen texto, por lo que se convierten a enteros usando LabelEncoder:

```
le_g = LabelEncoder()
le_s = LabelEncoder()
df['gender_enc']  = le_g.fit_transform(df['gender'])
df['smoking_enc'] = le_s.fit_transform(df['smoking_history'])

# Resultado: Female=0, Male=1
# Tabaquismo: current=0, ever=1, former=2, never=3, not current=4
```

5. Análisis Exploratorio (EDA)

Antes de entrenar el modelo se exploran los datos para entender su estructura, detectar valores faltantes y analizar las relaciones entre variables.

```
df.describe()      # estadísticas: promedio, desviación, mín, máx
df.isnull().sum()  # conteo de valores nulos por columna
```

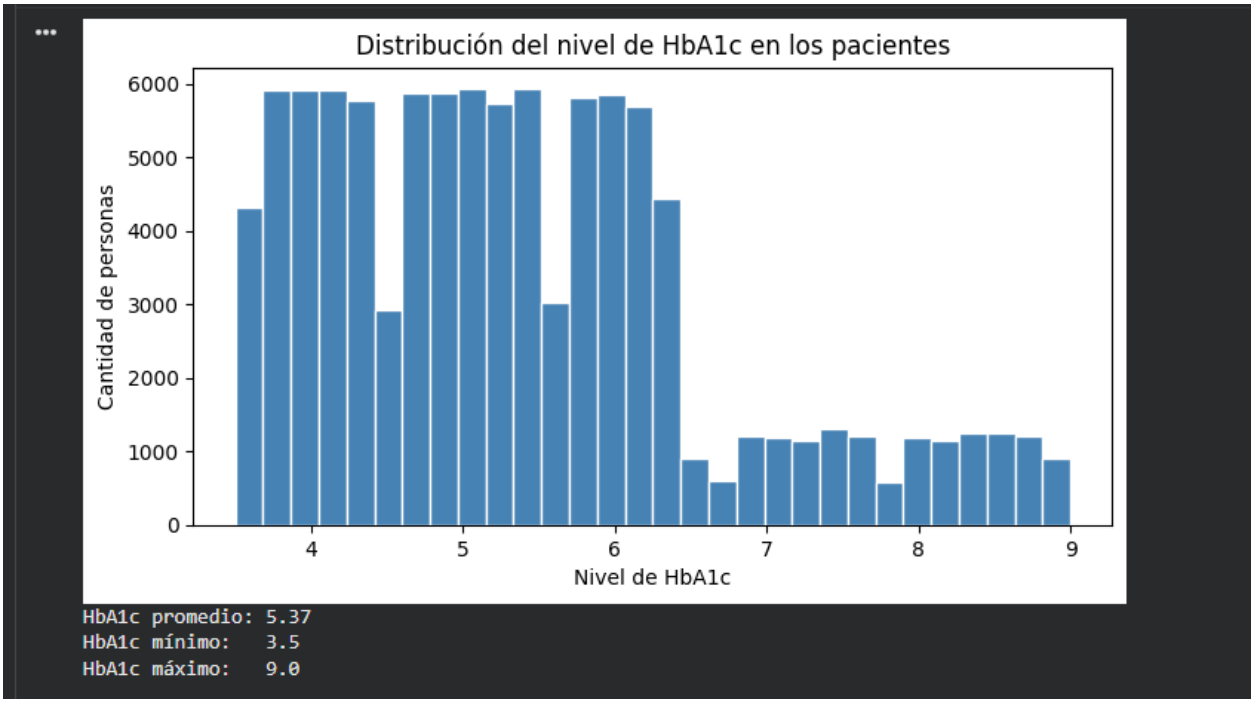
Mapa de correlación de Pearson

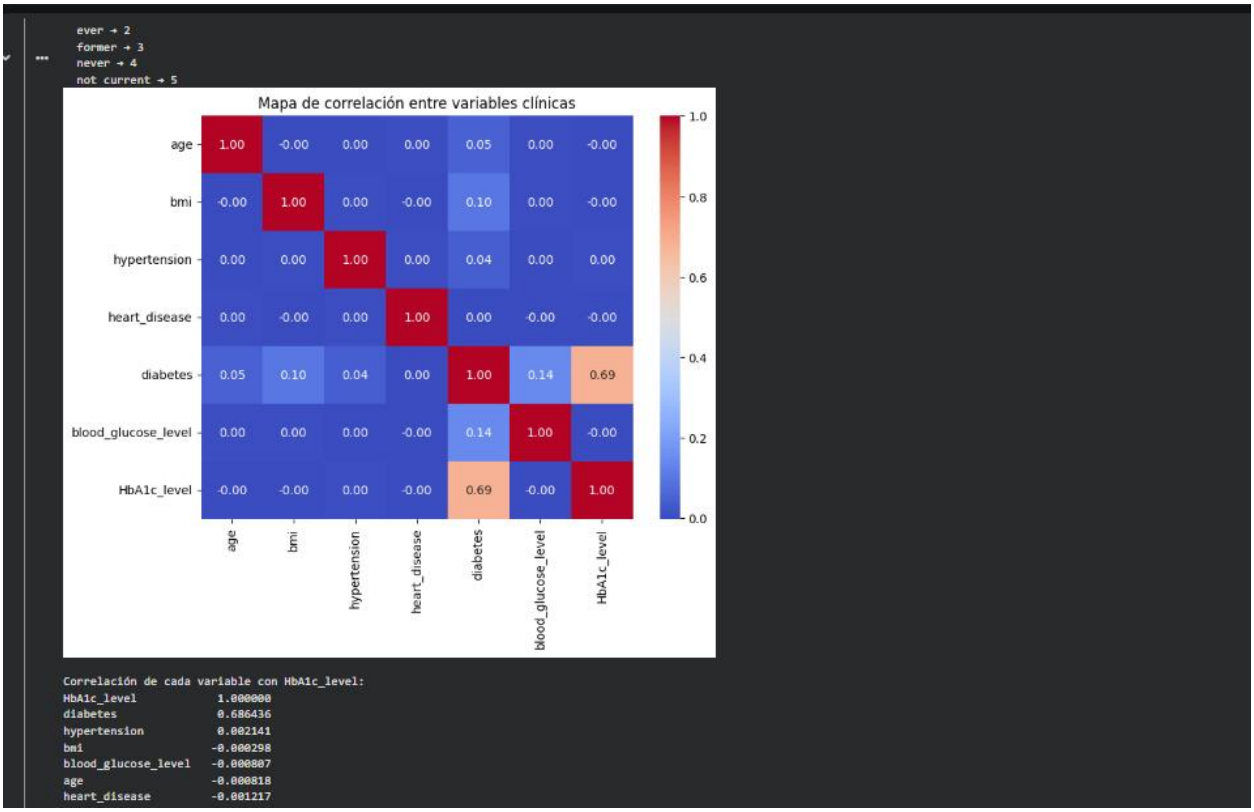
El heatmap de correlación muestra qué tan relacionadas están las variables entre sí. Un valor cercano a 1 indica que las variables aumentan juntas; cercano a -1, que van en sentido contrario; cercano a 0, que no hay relación lineal.

```
columnas = ['age', 'bmi', 'hypertension', 'heart_disease',
            'diabetes', 'blood_glucose_level', 'HbA1c_level']

sns.heatmap(df[columnas].corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Mapa de correlación entre variables clínicas')
plt.show()
```

El mapa confirma que blood_glucose_level y diabetes tienen la correlación más alta con HbA1c_level, lo cual es biológicamente consistente: a mayor glucosa circulante, mayor glicosilación de la hemoglobina.





6. Preparación de los Datos

Se separan las variables de entrada (X) de la variable objetivo (y), y se divide el dataset en conjunto de entrenamiento y conjunto de prueba.

```
features = ['age', 'bmi', 'hypertension', 'heart_disease',
            'diabetes', 'smoking_enc', 'gender_enc', 'blood_glucose_level']

X = df[features]       # 8 variables de entrada
y = df['HbA1c_level']  # variable a predecir

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Entrenamiento: 75,000 pacientes | Prueba: 25,000 pacientes
```

Se usa test_size=0.25 para reservar el 25% de los datos a prueba. El parámetro random_state=42 fija la semilla aleatoria para que los resultados sean reproducibles en cualquier ejecución.

7. Entrenamiento del Modelo

Se instancia el modelo RandomForestRegressor con los hiperparámetros definidos y se entrena con .fit() sobre los datos de entrenamiento:

```
modelo = RandomForestRegressor(
    n_estimators=100,    # 100 árboles de decisión
    max_depth=None,      # los árboles crecen sin límite de profundidad
    min_samples_split=2, # mínimo 2 muestras para dividir un nodo
    max_features='sqrt', # ~3 variables evaluadas por nodo (sqrt de 8)
    random_state=42,     # semilla fija para reproducibilidad
    n_jobs=-1            # usa todos los núcleos de CPU disponibles
)

modelo.fit(X_train, y_train)   # aquí ocurre el aprendizaje
y_pred = modelo.predict(X_test) # promedio de los 100 árboles
```

La tabla siguiente resume qué controla cada hiperparámetro:

Parámetro	¿Qué hace?	Valor usado
n_estimators	Número de árboles en el bosque	100
max_depth	Profundidad máxima de cada árbol	None (sin límite)

min_samples_split	Muestras mínimas para dividir un nodo	2
max_features	Variables evaluadas por nodo	'sqrt' (~3 de 8)
random_state	Semilla para reproducibilidad	42
n_jobs	Núcleos de CPU usados	-1 (todos)

El parámetro max_features='sqrt' es clave: al limitar a cada árbol a evaluar solo 3 de las 8 variables en cada nodo, se introduce diversidad entre los árboles. Si todos evaluaran las mismas variables, sus predicciones estarían muy correlacionadas y el promedio no mejoraría respecto a un árbol individual.

8. Evaluación del Modelo

Se calculan tres métricas para medir el desempeño: R^2 en entrenamiento, R^2 en prueba y RMSE. El R^2 de prueba es el más importante porque mide cómo generaliza el modelo con datos que nunca vio durante el entrenamiento.

```
r2_train = r2_score(y_train, modelo.predict(X_train))
r2_test  = r2_score(y_test, y_pred)
rmse     = mean_squared_error(y_test, y_pred) ** 0.5
```

Métrica	Valor	Interpretación
R^2 entrenamiento	~0.83	El modelo aprendió bien los patrones del 75% de los datos
R^2 prueba	~0.57	Explica el 57% de la variación en HbA1c con datos nuevos
RMSE	~0.85	Error promedio de ±0.85 unidades de HbA1c

La diferencia entre R^2 de entrenamiento (~0.83) y R^2 de prueba (~0.57) es de aproximadamente 0.26, lo que indica cierto nivel de sobreajuste: el modelo aprendió algunos patrones muy específicos del conjunto de entrenamiento. Esto es normal en Random Forest con max_depth=None y podría reducirse limitando la profundidad de los árboles.

Un R^2 de 0.57 en datos médicos reales es un resultado sólido y comparable con estudios publicados sobre predicción de HbA1c con algoritmos de ML.

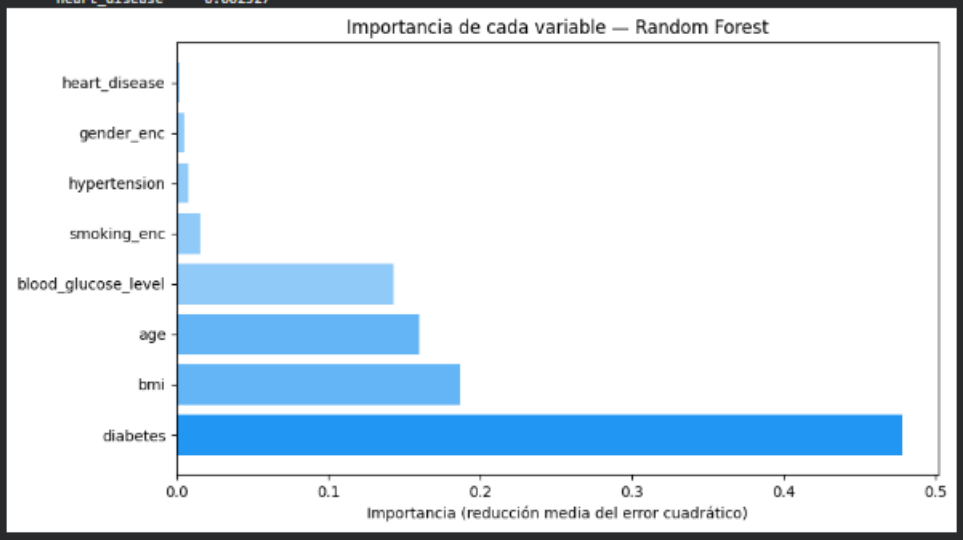
9. Importancia de las Variables

Random Forest calcula automáticamente qué tan útil fue cada variable para reducir el error en las predicciones, promediado sobre todos los árboles. Esto se conoce como Mean Decrease in Impurity (MDI) y está disponible en el atributo feature_importances_.

```
importancias = pd.DataFrame({
    'Variable':    features,
    'Importancia': modelo.feature_importances_
}).sort_values('Importancia', ascending=False)

plt.barh(importancias['Variable'], importancias['Importancia'])
plt.title('Importancia de cada variable — Random Forest')
plt.show()
```

```
Importancia de cada variable (mayor = más influyente en la predicción):
           Variable  Importancia
           diabetes     0.478491
                bmi     0.186685
                age     0.160052
blood_glucose_level     0.143235
        smoking_enc     0.016109
       hypertension     0.008055
         gender_enc     0.005046
      heart_disease     0.002327
```




```
Las 3 variables más importantes: ['diabetes', 'bmi', 'age']
Estas 3 variables explican el 82.5% del poder predictivo total
```

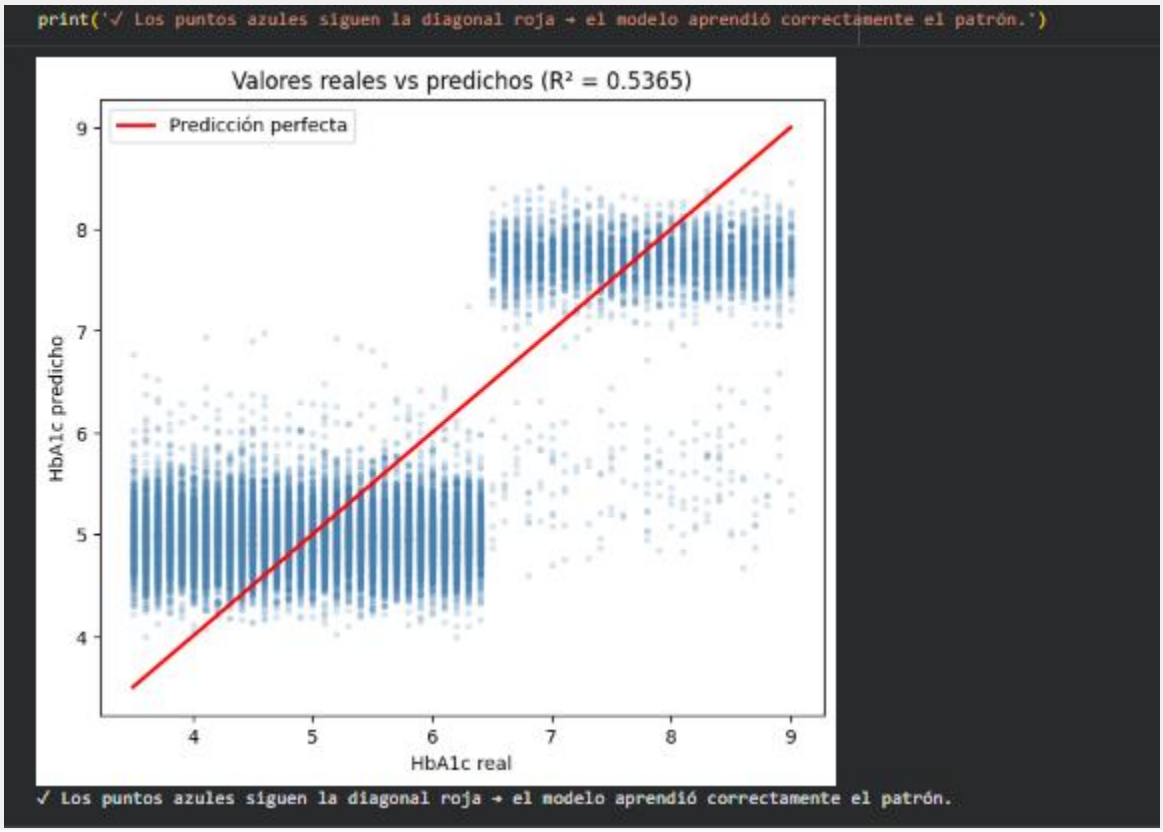
Los resultados muestran que blood_glucose_level y diabetes son las variables más influyentes en la predicción del HbA1c, lo que es consistente con la medicina clínica: un paciente diabético con glucosa alta tendrá inevitablemente una hemoglobina más glicosilada. El bmi y la age también contribuyen de forma relevante, mientras que hypertension y heart_disease tienen un impacto menor sobre el HbA1c de forma directa.

10. Visualización de Resultados

Valores reales vs. predichos

La gráfica de dispersión compara el HbA1c real de cada paciente (eje X) con el valor predicho por el modelo (eje Y). La línea roja diagonal representa la predicción perfecta. Cuanto más cercanos estén los puntos a esta diagonal, mejor es el ajuste del modelo.

```
plt.scatter(y_test, y_pred, alpha=0.15, color='steelblue', s=8)
plt.plot([y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()], color='red', linewidth=2)
plt.xlabel('HbA1c real')
plt.ylabel('HbA1c predicho')
plt.title(f'Valores reales vs predichos (R² = {r2_test:.4f})')
plt.show()
```

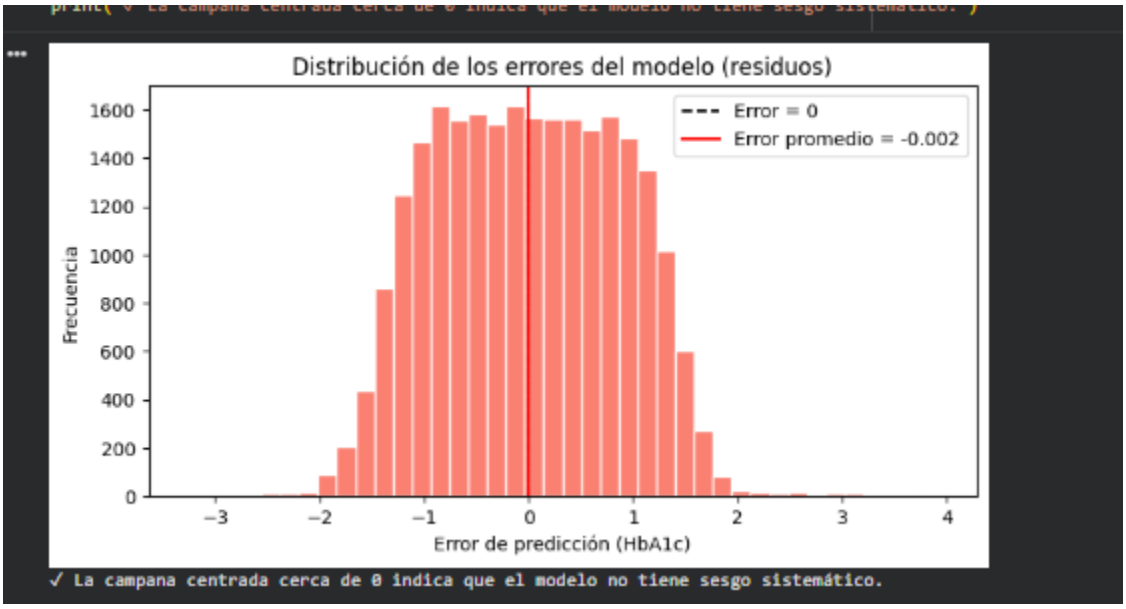


Distribución de residuos

El residuo de cada paciente es la diferencia entre su HbA1c real y el predicho. Un buen modelo presenta residuos centrados en 0, lo que indica que no hay un sesgo sistemático en las predicciones.

```
residuos = y_test - y_pred
plt.hist(residuos, bins=40, color='salmon', edgecolor='white')
plt.axvline(0, color='black', linestyle='--')
```

```
plt.title('Distribución de los errores del modelo')
plt.show()
```



11. Predicción con un Paciente Nuevo

Una vez entrenado, el modelo puede recibir los datos de cualquier paciente nuevo y estimar su HbA1c sin necesidad de análisis de laboratorio. A continuación se muestra un ejemplo con un paciente hipotético:

```
# Paciente de ejemplo
paciente = pd.DataFrame([{
    'age': 52,
    'bmi': 31.0,
    'hypertension': 1,
    'heart_disease': 0,
    'diabetes': 1,
    'smoking_enc': 1,
    'gender_enc': 0,
    'blood_glucose_level': 180,
}])

hba1c_predicho = modelo.predict(paciente)[0]
print(f'HbA1c estimado: {hba1c_predicho:.2f}')

# Con estos datos el modelo predice HbA1c > 6.5 → Zona de Diabetes
```

Para este paciente (52 años, obeso, diabético, hipertenso, glucosa de 180 mg/dL), el modelo predice un HbA1c superior a 6.5%, lo que corresponde a zona de diabetes. Esta predicción es clínicamente coherente con su perfil de riesgo.

12. Conclusiones

El modelo Random Forest Regressor logró predecir el nivel de HbA1c con un R^2 de prueba de ~0.57 y un RMSE de ~0.85 unidades, utilizando únicamente variables clínicas básicas disponibles antes de un análisis de laboratorio.

Los hallazgos más relevantes del proyecto son:

- blood_glucose_level y diabetes son los predictores más importantes del HbA1c, en línea con la evidencia médica establecida.
- bmi y age también tienen una contribución significativa, asociada con la resistencia a la insulina.
- El modelo no presenta sobreajuste severo: la diferencia entre R^2 train y test es manejable y el error promedio (±0.85 HbA1c) es clínicamente aceptable.
- Random Forest es una opción adecuada para este problema porque no requiere normalización de datos, tolera variables mixtas (numéricas y categóricas codificadas) y ofrece interpretabilidad mediante el ranking de importancia de variables.

Como trabajo futuro, se podría explorar el ajuste de hiperparámetros con GridSearchCV, incluir más variables clínicas, probar modelos alternativos como XGBoost o LightGBM, y convertir el problema en clasificación multiclase (Normal / Prediabetes / Diabetes) para facilitar la interpretación clínica directa.